**ASTRA RESEARCH MONOGRAPH**

# Research vision, Phase 1 charter integration, procedural intelligence compiler, safety model, and experimental roadmap

**Defining thesis**
**Astra compiles verified procedural capabilities from experience.**
The model is temporary reasoning support. The acquired procedural intelligence belongs to Astra.

**Research blueprint v1.1 - Phase 1 normative revision**
Repository evidence snapshot: branch feature/chat-native-approval, commit 9d7b63a41cf4
Prepared 29 July 2026 — local-first, Python-first, hardware-constrained research programme
The Phase 1 charter is normative and overrides less precise language in this monograph.

# Document status and reading guide

This monograph is both an architecture specification and a research proposal. It reconstructs the reasoning that led from the original Astra coding-assistant concept to the refined idea of verified procedural capability compilation. It is intentionally explicit about evidence status. Statements marked **implemented** are grounded in the inspected repository. **Emerging** elements exist in partial or experimental form. **Proposed** elements are architectural designs. **Research** elements are hypotheses that require controlled experiments.

IMPLEMENTED EMERGING PROPOSED RESEARCH

**Claim discipline**

Astra is not presented as the first system to extract skills, learn workflows, or reuse trajectories. Related work already explores those topics. The proposed contribution is the continuous compilation of real software-engineering outcomes into deterministic, executable, model-independent procedures that remain subordinate to a fixed safety kernel.

**Normative precedence**

This blueprint explains Astra's vision, architecture, evidence snapshot, and research programme. The Phase 1 charter freezes definitions and epistemic rules. Where the two differ, the charter controls. Changes to capability identity, experience semantics, attribution, preference governance, learning authority, or transfer evaluation require an explicit charter amendment.

## Executive abstract

Astra began as an attempt to create a useful local coding assistant on a laptop with an NVIDIA RTX 3050 Laptop GPU and 4 GiB of VRAM. The hardware can run small coding models, but it cannot reproduce the broad internal knowledge and inference capacity of frontier-scale language models. Live work with Qwen2.5-Coder 1.5B made the limitation concrete: the model could write a requested function, yet whole-file rewriting could omit correct existing functions. A deterministic append operation succeeded because the system reduced the model's responsibility to the fragment it handled well.

That lesson changes the research question. Astra should not ask a small model to imitate a large model. It should build intelligence in software: repository graphs, typed intent, evidence packages, semantic operations, strict state machines, isolated validation, outcome ledgers, strategy models, and eventually a capability compiler. The compiler observes verified trajectories, abstracts repeated procedures into a restricted intermediate representation, tests them through simulation and replay, benchmarks them on held-out tasks, and recommends only qualified artifacts for governed promotion into a versioned capability library. The broader Procedural Intelligence Compiler also preserves applicability, invariants, verification contracts, attribution, failure evidence, and recomputable performance assessments.

The central falsifiable hypothesis is that Astra, after 1,000 chronological engineering episodes across a declared number of repositories, should solve more held-out tasks with fewer model calls, retries, and clarifications than its initial version while using the same model, prompts, hardware, and safety kernel. If performance does not improve under those controls, the capability-compilation hypothesis is rejected or revised.

# Phase 1 normative summary

**Frozen research identity**

**Astra is an evidence-governed procedural intelligence architecture for software engineering.** Its learned state may improve retrieval, ranking, applicability, clarification, failure prediction, and composite capabilities. It may not learn or rewrite authority, permissions, workspace boundaries, artifact integrity, approval, isolation, idempotency, or verification authority.

## N.1 Normative definitions

| Term | Frozen Phase 1 definition |
|---|---|
| Software-engineering intelligence | Measured improvement in held-out engineering decisions and outcomes under fixed resources, authority, and model support as verified experience accumulates. |
| Observation | A provenance-preserved event or artifact. It is not automatically evidence. |
| Evidence | An observation interpreted for or against an explicit claim through a versioned evidence vocabulary. |
| Experience | An immutable episode joining pre-decision context, available features, alternatives, intervention, execution, verification, user observations, outcome, and attribution hypotheses. |
| Capability | A versioned procedural artifact C=(A,P,I,V): applicability, procedure, invariants/safety argument, and verification contract. |
| Learning | A controlled, evidence-backed change to derived decision state that may improve future outcomes without changing the fixed authority kernel. |
| Transfer | Reuse of an unchanged capability on held-out context that differs along declared capability-relevant dimensions while P, I, and V remain valid. |

## N.2 Capability identity

Two implementations belong to one capability only while they share one causal procedure family, one safety argument, and one verification contract. Source similarity, trajectory embeddings, or a failed unification search cannot decide identity. Applicability predicates may select and parameterise a procedure; they may not hide an alternative procedure.

| Evolution decision | Meaning | Required discipline |
|---|---|---|
| Refine | Narrow or sharpen A while preserving P, I, and V. | Recompute inherited statistics from immutable episodes under the new predicate. |
| Split | Create a probationary child for a distinct causal procedure or safety/verification account. | Require a minimal distinguishing witness and independent transfer evidence. |
| Compose | Build a composite from existing capabilities. | Require child contracts plus an emergent integration verifier. |

# Phase 1 normative summary — evidence and governance

## N.3 Epistemic rules

• **Feedback is an observation. Attribution is a hypothesis.** Rejection does not prove plan defect, preference, convention mismatch, interaction cost, or presentation failure.

• **Failed bounded unification leaves identity unresolved.** It may justify a provisional split candidate, never an automatic permanent split.

• **Outcome history belongs to immutable experiences.** Capability statistics are derived and recomputed for the current capability and predicate versions.

• **One severe failure may veto promotion.** Invariant or safety violations have different evidential force from infrastructure failures or correct abstention.

• **User preferences have the lowest authority.** They are testable output constraints, quarantined from capability compilation, confirmed before first influence, visible, scoped, and deletable as derived views.

• **Held-out means uninvolved.** A target is not held out if it influenced synthesis, predicate refinement, operation vocabulary, verifier design, thresholds, or evolution decisions.

## N.4 Recomputable assessments

Immutable experiences preserve the raw feature values and vocabulary versions required to evaluate future predicates and transfer profiles that did not yet exist. Capability statistics, applicability, preference views, and transfer strata are projections, not historical truth. Every derived assessment records its version, original classification, current recomputation, and supersession status.

## N.5 Compiler authority boundary

| Compiler may | Compiler may not |
|---|---|
| Construct candidate DSL and applicability hypotheses. | Write arbitrary Python into the trusted kernel. |
| Mine positive and negative experience. | Treat public examples as trusted production procedures. |
| Recommend refine, split, compose, promotion, degradation, or revocation. | Grant production authority or approve its own recommendation. |
| Request simulation, replay, and held-out evaluation through trusted services. | Redefine a verifier after seeing held-out results. |
| Detect recurring ambiguity in the atomic-operation vocabulary. | Silently change the operation vocabulary. |

**Experimental unit**
In this monograph, a **project count** refers to a canonical software-engineering episode: one bounded user objective executed through one canonical Astra project lifecycle. Results must report both the number of episodes and the number of independent repositories. Multiple episodes from one repository are not treated as statistically independent repository-transfer cases.

**Scientific test**

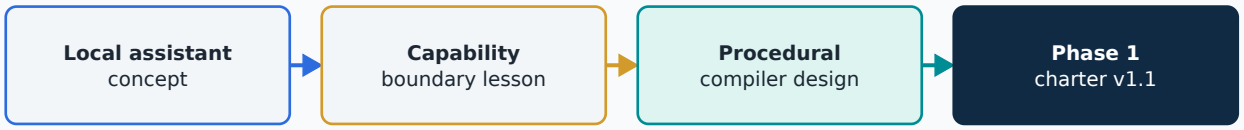
After 1,000 chronological engineering episodes across a declared number of repositories, Astra must improve repository-disjoint transfer, correct abstention, efficiency, and calibration relative to static, memory-only, and ranking-only baselines. Memorising exact patches, increasing capability count, or narrowing coverage without reporting it is not intelligence.

# Contents

Chapter 1

# Why Astra Changed Direction

The project did not abandon its original ambition. It changed the unit of intelligence from a monolithic model to a verified software system.




Astra Next Research Blueprint v1.1 — Phase 1 Normative Revision

Figure 1.1 — The evolution of Astra’s research question.

## 1.1 The original constraint

The starting requirement was unusually strict: build a local coding assistant that feels responsive and competent on a laptop with approximately 32 GiB of installed physical RAM, two SSDs, and a 4 GiB RTX 3050 Laptop GPU. Everything should remain local. Long training, model downloads during operation, uncontrolled shell execution, and cloud dependence were unacceptable. The initial instinct was to combine a small language model with specialist deep-learning models, machine-learning classifiers, Python rules, workers, retrieval, and memory so that no single component carried the full cognitive load.

That instinct was directionally correct, but early architecture accumulated overlapping subsystems: legacy SLM routes beside canonical local-AI services, multiple orchestration paths, old and new retrieval systems, broad specialist concepts, and a growing API surface. The problem was not a lack of ideas. It was the absence of a sufficiently narrow definition of what each layer was allowed to decide.

## 1.2 Hardware is a design input, not an embarrassment

| Resource | Available system | Architectural consequence |
|---|---|---|
| GPU | RTX 3050 Laptop, 4 GiB VRAM | One bounded local GPU workload; small quantized model; explicit admission. |
| Installed physical RAM | 33,962,164,224 bytes (31.63 GiB) | Host capacity measured through Windows on 29 July 2026. |
| Windows-visible RAM | 33,962,164,224 bytes; 17,610,481,664 free | Free memory is a time-varying observation, not a machine constant. |
| WSL-visible RAM | Runtime-dependent; capture per experiment | Never infer WSL limits from installed host RAM. |
| Storage | 1 TB + 512 GB SSD | Immutable artifacts, benchmark fixtures, snapshots, and model files are feasible. |
| Runtime | Python-first, local-only | Prefer ASTs, rules, subprocess isolation, and small learned rankers. |
| Model episode | Qwen2.5-Coder 1.5B via Ollama | Decisive failure involved 1.5B; the architectural conclusion is model-independent. |
| Later observation | Qwen2.5-Coder 3B, operator-reported | Improved some structured generation; did not remove bounded responsibility or deterministic integration. |

**Measurement provenance**
Installed and Windows-visible memory were captured from the host operating system on 29 July 2026 (Windows CIM/OS performance counters / system information reporting total physical memory 33,962,164,224 bytes). WSL-visible memory, available memory, GPU allocation, and free VRAM are runtime observations and must be captured separately for each experiment together with the command, timestamp, active processes, and benchmark configuration. Example capture commands: `wmic ComputerSystem get TotalPhysicalMemory`; in WSL `free -h` and `nvidia-smi --query-gpu=memory.total,memory.free --format=csv`.

## 1.3 What live synthesis taught us

The decisive engineering episode was an apparently trivial calculator task. Whole-file synthesis asked the 1.5B model to preserve existing functions while adding another. The model implemented the new function correctly but dropped valid existing code. Deterministic semantic guards rejected the proposal. The successful redesign introduced a bounded append-python-symbol strategy: the model wrote only the new function, while Astra preserved and extended the existing file. The safety system did not become weaker; the proposal construction became easier for the model and easier to verify.

**General lesson**
Reducing the model’s responsibility to the fragment it handles well is not a limitation. It is the design. Deterministic software carries the structural reasoning. The model fills a small, well-specified slot. This principle governs the entire Astra Next architecture.

Chapter 2

# The Research Question

Astra’s research question is whether verified procedural experience produces measurable improvement in held-out software-engineering outcomes.

## 2.1 From model imitation to procedural compilation

The original question was: can a small local model provide competent coding assistance? The revised question is: can a software system accumulate verified procedural intelligence that reduces dependence on model size over time? These are different questions with different hypotheses, measurements, and falsification criteria.

Model imitation asks whether a 1.5B model can approximate a 70B model. The answer is no for broad tasks. Procedural compilation asks whether recurring verified task procedures, once extracted and tested, allow the same small model to succeed more often on the tasks those procedures cover. This is a more modest and more tractable claim.

## 2.2 The central hypothesis

> **Falsifiable hypothesis**
> Astra, after accumulating chronological experience, should improve held-out task success, capability-relative transfer, correct abstention, model-call efficiency, and calibration relative to static, memory-only, and ranking-only baselines under identical model, hardware, prompts, and safety kernel. If no improvement appears, the compilation hypothesis is rejected or revised.

## 2.3 What improvement must not mean

Accumulating more capability files is not improvement. Narrowing applicability predicates to raise conditional success while hiding coverage loss is not improvement. Memorising test fixtures is not improvement. Increasing model calls while storing them as experience is not improvement. Every gain must be accompanied by held-out evidence, coverage, transfer stratum, and a comparison against memory-only access to the same trajectory data.

## 2.4 The relationship between safety and learning

The safety kernel is fixed not because safety is unimportant to research but because it is the prerequisite for trustworthy measurement. An Astra that silently widens its own authority makes improvement claims uninterpretable. A fixed authority boundary makes capability gains visible and attributable.

Chapter 3

# The Layered Intelligence Model

Astra separates what must remain deterministic from what may be learned. Each layer has a precise charter.

## 3.1 Why layers matter

**1. Safety and authority.** Non-negotiable fixed boundary. Governs all lifecycle transitions, approvals, isolation, integrity, and verification authority.

**2. Procedural intelligence.** Compiled capability artifacts. Must survive simulation, replay, and held-out transfer before activation.

**3. Decision and ranking.** Learned selection over allowlisted strategies. May improve with experience. Cannot introduce new authority.

**4. Repository intelligence.** Deterministic structural knowledge. Freshness, provenance, and scope are mandatory attributes.

**5. Semantic interface.** Translation of natural language into bounded typed hypotheses. Never directly mutates state.

## 3.2 Intelligence layers

| Layer | What changes | What must remain fixed |
|---|---|---|
| Semantic | Mappings from language to typed hypotheses; clarification policy. | No direct authority over project state. |
| Repository intelligence | Profiles, relevance scores, evidence contracts. | Canonical file identities and path safety. |
| Decision | Strategy rankings and failure probabilities. | Allowlisted actions and transition guards. |
| Procedural | Compiled capability artifacts and applicability models. | Trusted atomic operation implementations. |
| Safety | Nothing learned automatically. | Scope, approval, isolation, integrity, verification. |

## 3.3 Why a fixed safety kernel matters

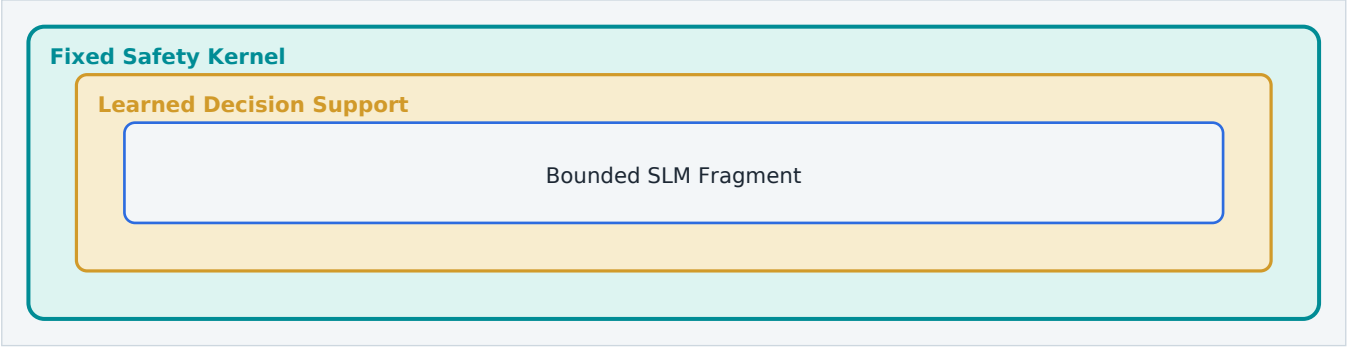



Figure 3.1 — Learned intelligence is contained inside a fixed authority boundary.

The SLM is a bounded synthesis tool that produces untrusted fragments for the semantic layer and typed validator feedback. It is not the project lifecycle authority, the verifier, the memory database, or the planner of unrestricted actions. Replacing Qwen with another local model should change generation quality, not erase Astra’s accumulated capabilities.

Chapter 4

# Current Astra: Evidence and Baseline

Astra already contains a strong canonical control plane, local-AI boundary, worker, isolation layer, retrieval, deterministic analysis, and an emerging decision/outcome spine.

## 4.1 Repository snapshot

| Evidence item | Observed value | Interpretation |
|---|---|---|
| Branch | feature/chat-native-approval | Active local development branch. |
| Commit | 9d7b63a41cf4 | Reliable add-function synthesis and self-correcting retry. |
| Worktree state | Dirty: 102 porcelain entries | Counts include uncommitted local work; they are not a clean-commit measurement. |
| Test files | 190 test_*.py; 191 total Python files | Measured under tests/ on 29 July 2026. |
| Backend test functions | 1,508 | Substantial safety and behavior coverage. |
| Deterministic benchmark | 40/40 passed; phase0.v1 | Artifact generated 28 July 2026 at 06:37:48 UTC under Python 3.12.3. |
| Canonical model boundary | LocalAIService | Admission, provider execution, provenance, and structured output. |
| Lifecycle authority | ProjectControlPlane | Exact transitions, approvals, attempts, artifacts, and idempotency. |

**Snapshot commands**

`git status --porcelain=v1`; `rg --files tests -g 'test_*.py'`; `rg -n '^\s*(async\s+)?def\s+test_' tests -g '*.py'`. Benchmark evidence comes from `benchmarks/results/baseline_2026-07-28.json`. Skipped-test counts are not implied because the pytest suite was not executed for this document revision.

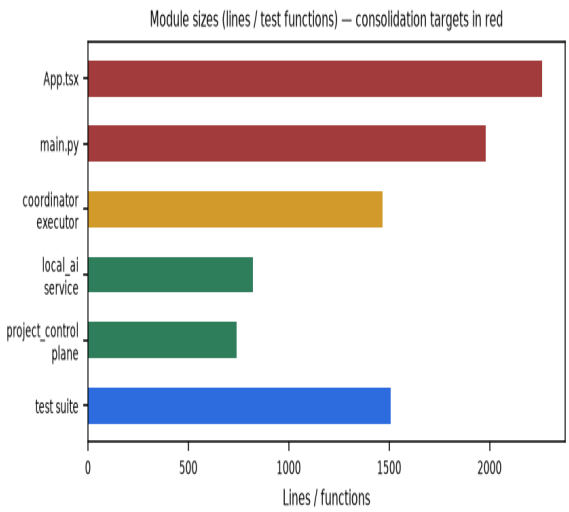



Figure 4.1 — Large files reveal consolidation work that should precede broad expansion.

## 4.2 Implemented strengths

- Exact project, conversation, actor, revision, manifest, state-version, and idempotency bindings.
- Immutable plan, scope, approval, execution-attempt, event, and verifier evidence records.
- Durable worker ownership separated from browser and request processing.
- Fail-closed Docker execution with network disabled, non-root identity, pinned image, and bounded resources.
- Workspace registration and path allowlisting instead of arbitrary browser filesystem access.
- LocalAIService as canonical admission and generation authority.
- Strict provider readiness, model availability, GPU exclusivity, and typed local-AI failures.
- AST analysis, deterministic fixes, scaffolding blueprints, project retrieval, and focused benchmark fixtures.

## 4.3 Emerging research spine

The current worktree contains an advisory DecisionLayer, an append-only DecisionOutcomeStore, rules-first playbook selection, retrieval integration, and conservative memory-based strategy choice. The ranker is explicitly shadow-only. These are valuable beginnings, but they are not yet a capability compiler. They should be treated as instrumentation and baseline policy rather than proof of continual learning.

## 4.4 Complexity debt

Several critical files have grown beyond practical review size: the main backend entry point approximately 1,980 lines, coordinator execution approximately 1,466 lines, and the main React component approximately 2,262 lines. Legacy and canonical subsystems coexist. A senior engineering plan should consolidate to one model boundary, one project workflow, one retrieval system, one worker, and a smaller composition root before the research layer becomes production-critical.

**Historical-document caution**

Some repository documents describe earlier gaps that have since been closed. For example, the task-family specification originally described add_function_to_module as a 0% baseline gap, while the latest inspected phase0.v1 result reports all three cases passing. The monograph uses timestamped evidence rather than assuming every historical document describes the current code.

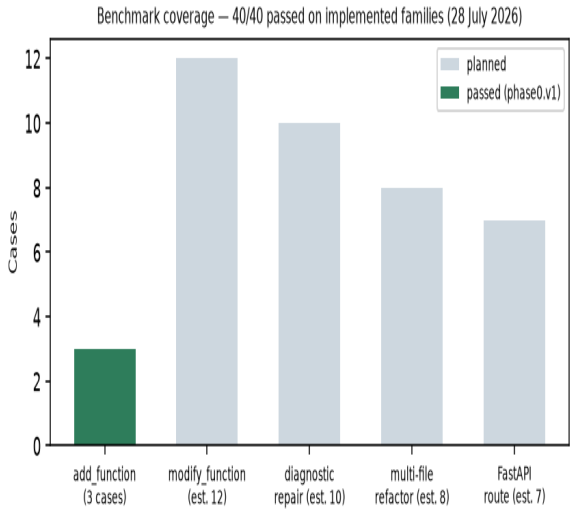



Figure 4.2 — Current deterministic benchmark composition and latest result.

Chapter 5

# Target System Architecture

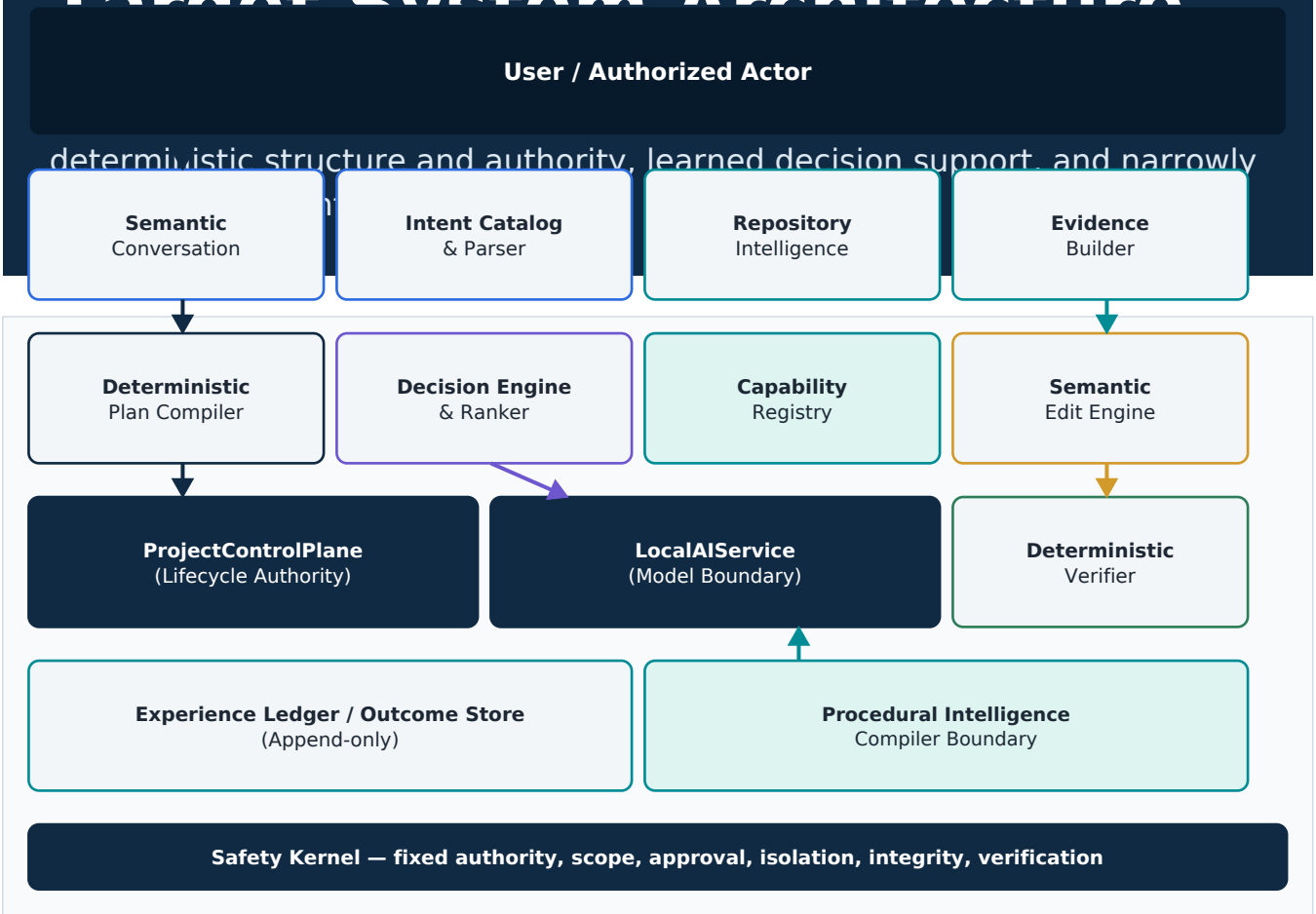



Figure 5.1 — Full target architecture and authority flow.

**Diagram semantics**
Navy arrows indicate canonical authority flow; teal arrows indicate promoted capability or evidence flow; purple arrows indicate advisory model interaction; gold arrows indicate execution and validation flow. Learned or model-generated outputs never directly mutate canonical state.

## 5.1 End-to-end request flow

1. Conversation is translated into one or more typed intent hypotheses.
2. Repository intelligence constructs a task-specific evidence package.
3. The deterministic plan compiler maps the intent to bounded work units.
4. The decision engine ranks allowlisted capabilities and estimates uncertainty.

**5.** A trusted capability executes deterministic operations or requests a tiny model-generated fragment.

**6.** Scope, schema, AST, semantic, integrity, and policy guards validate the proposal.

**7.** Focused tests run automatically only in an allowlisted isolated validation profile.

**8.** The user receives the plan, patch, retry history, and evidence; patch application remains explicitly approved.

**9.** Post-apply verification generates immutable evidence and a canonical outcome.

**10.** The experience engine records the decision context and outcome for later compilation.

## 5.2 One canonical owner per responsibility

| Responsibility | Canonical owner | Forbidden duplication |
|---|---|---|
| Lifecycle | ProjectControlPlane | Route-local or frontend state transitions. |
| Model execution | LocalAIService | Direct provider calls from feature modules. |
| Project coordination | Canonical coordinator + worker | Second autonomous job authority. |
| Retrieval | Project-bound retrieval service | Unbound generic RAG in production path. |
| Capabilities | Versioned capability registry | Prompt-only hidden procedures. |
| Verification | Deterministic verifier | Model claims treated as evidence. |
| Experience | Append-only outcome ledger | Mutable chat summaries as truth. |

## 5.3 Deployment topology

Local development uses four explicit processes: Ollama, FastAPI, the continuous project worker, and the frontend. The backend and worker consume the same reviewed environment configuration. Astra does not start Ollama, pull models, install packages, or build execution images during a project request. GPU scheduling remains exclusive unless a stricter mechanism is introduced.

```
Operator starts Ollama
        |
        +-- FastAPI backend  ------ SQLite canonical state
        |
        +-- Continuous worker ---- coordinator intents / bounded execution
        |
        +-- React frontend  ------- canonical read model and approvals

LocalAIService -> provider readiness -> GPU admission -> one model workload
Project worker -> disposable Docker snapshot -> focused validation
```

## 5.4 Why not a swarm of agents?

Layer cooperation should use typed contracts rather than conversational agents. Retrieval returns file identities and evidence artifacts. Planning returns work units. Validation returns typed results. A small model may be called inside a bounded stage, but inter-layer messages remain inspectable data. This design reduces token use, prevents circular conversation, and makes replay possible.

Chapter 6

# Component Specifications

Every layer is defined by its decision, inputs, outputs, algorithm, failure behavior, and benchmark. Vague component names are not enough.

| Semantic Conversation Layer | PROPOSED |
|---|---|
| **Purpose** | Presents a Codex-like conversational experience while translating natural language into bounded intent hypotheses and explaining canonical state. |
| **Inputs** | User message, conversation binding, selected project, active project state, known vocabulary. |
| **Outputs** | Typed intent candidates, missing-slot questions, safe narration, no lifecycle mutation. |
| **Method** | Small SLM extraction plus deterministic schema validation, repository-aware vocabulary, and clarification thresholds. |
| **Failure mode** | Ambiguity, invalid schema, or unavailable model produces clarification or deterministic fallback; never an invented action. |
| **Benchmark** | Intent accuracy, clarification precision, user correction rate, latency, model-call count. |

| Intent Catalog and Parser | EMERGING |
|---|---|
| **Purpose** | Defines the finite set of tasks Astra understands and maps messages into catalog entries with typed slots. |
| **Inputs** | Normalized text, repository context, existing conversation decisions. |
| **Outputs** | IntentResolution with task family, slots, confidence, alternatives, and clarification reason. |
| **Method** | Rules-first parsing, optional semantic model, exact slot validators, and catalog versioning. |
| **Failure mode** | Unknown or conflicting requests resolve to clarification_needed rather than generic project execution. |
| **Benchmark** | Macro/micro F1, unknown rejection accuracy, slot exact match, calibration. |

| Repository Intelligence Engine | IMPLEMENTED |
|---|---|
| **Purpose** | Builds deterministic structural knowledge before any model reasons about code. |
| **Inputs** | Allowlisted root, manifests, ASTs, imports, configuration, test discovery, diagnostics. |
| **Outputs** | Repository profile, symbol graph, dependency edges, framework facts, evidence references. |

| | |
|---|---|
| **Method** | Bounded scanners, AST parsing, configuration readers, graph construction, content hashes. |
| **Failure mode** | Limits, unsafe paths, symlinks, incomplete manifests, or parsing uncertainty remain explicit. |
| **Benchmark** | Localization recall, graph accuracy, scan latency, manifest completeness, false positives. |

| **Evidence Builder** | **IMPLEMENTED** |
|---|---|
| **Purpose** | Compresses repository structure into the minimum task-relevant evidence package. |
| **Inputs** | Typed intent, repository graph, retrieval candidates, file identities, existing tests. |
| **Outputs** | Versioned evidence artifact with approved paths, source excerpts, symbols, hashes, and provenance. |
| **Method** | Deterministic filters first, BM25/symbol ranking next, optional learned reranking last. |
| **Failure mode** | Missing or stale evidence blocks high-risk work; the model never fills absent source facts. |
| **Benchmark** | Evidence recall@k, token/character budget, stale-evidence rate, downstream success. |

| **Deterministic Plan Compiler** | **PROPOSED** |
|---|---|
| **Purpose** | Turns typed intent and evidence into bounded work units rather than asking a model to invent a workflow. |
| **Inputs** | Intent schema, evidence artifact, capability registry, project policy. |
| **Outputs** | Immutable plan revision with work units, scope, expected artifacts, and validation recipe. |
| **Method** | Rule-based mapping from task family to capability graph; unresolved choices become clarification. |
| **Failure mode** | No matching safe capability yields capability_gap, not free-form autonomy. |
| **Benchmark** | Plan validity, unnecessary step count, scope precision, approval rejection rate. |

| **Decision Engine and Strategy Ranker** | **EMERGING** |
|---|---|
| **Purpose** | Ranks allowlisted strategies while preserving deterministic default behavior and explicit uncertainty. |
| **Inputs** | Task family, repository profile, evidence quality, outcome summaries, resource state. |
| **Outputs** | Ranked strategies, predicted success, expected cost, explanation, clarification recommendation. |
| **Method** | Rules baseline; memory summary; later logistic regression, boosted trees, or contextual bandit in shadow mode. |
| **Failure mode** | Ranker cannot introduce a new strategy or bypass gates; low confidence falls back to rules or clarification. |
| **Benchmark** | Top-1 success, regret, calibration error, unsafe-selection rate, latency. |

| **Capability Registry** | **PROPOSED** |
|---|---|
| **Purpose** | Stores atomic and compiled procedures with versions, provenance, applicability, validators, and lifecycle state. |

| | |
|---|---|
| **Inputs** | Manually trusted operations and promoted compiled capability artifacts. |
| **Outputs** | Deterministic lookup by task, language, framework, evidence contract, and policy. |
| **Method** | Content-addressed declarative packages compiled to trusted Python operation handlers. |
| **Failure mode** | Unknown DSL version, missing validator, revoked dependency, or ambiguous match fails closed. |
| **Benchmark** | Coverage, reuse, overlap, activation precision, dependency health. |

| **Semantic Edit Engine** | EMERGING |
|---|---|
| **Purpose** | Applies language-aware operations such as append function, replace function, add import, and add test. |
| **Inputs** | File identity, expected AST shape, requested symbol contract, bounded generated fragment. |
| **Outputs** | Proposed file artifact and exact semantic delta. |
| **Method** | AST/CST parsing, typed operation preconditions, formatting, before/after semantic diff. |
| **Failure mode** | Wrong symbol, signature, extra top-level nodes, stale hash, or out-of-scope path is rejected. |
| **Benchmark** | Symbol precision, semantic preservation, scope violation rate, formatting correctness. |

| **Project Control Plane Worker** | IMPLEMENTED |
|---|---|
| **Purpose** | Executes coordinator intents in a bounded, isolated, and idempotent worker loop. |
| **Inputs** | Coordinator intent, project bindings, approved capabilities, resource limits. |
| **Outputs** | Transition records, artifacts, and validation results linked to the lifecycle authority. |
| **Method** | Canonical transition coverage, concurrency convergence, deterministic replay under given intent. |
| **Failure mode** | Stale binding, concurrency conflict, isolation failure, or resource exhaustion remains a typed non-success. |
| **Benchmark** | Transition coverage, concurrency convergence, replay behavior, stale-binding rejection. |

| **Capability Compiler** | RESEARCH |
|---|---|
| **Purpose** | Compiles recurring verified procedures into deterministic, executable, model-independent capability artifacts. |
| **Inputs** | Canonical trajectories, outcome ledger, repository profiles, trusted atomic operation vocabulary. |
| **Outputs** | Candidate DSL artifact, applicability predicate, evidence contract, validators, benchmark dossier. |
| **Method** | Pattern mining, anti-unification, parameterization, type checking, simulation, replay, held-out benchmarking. |
| **Failure mode** | No direct kernel writes; unsafe operations, poor transfer, redundancy, or weak evidence prevent promotion. |
| **Benchmark** | Promotion precision, held-out utility, reuse, regressions, deprecation, net verified capability gain. |

| **Capability Library** | **RESEARCH** |
|---|---|
| **Purpose** | Maintains production, experimental, degraded, deprecated, and revoked procedural artifacts. |
| **Inputs** | Promoted compiler artifacts and manually authored atomic capabilities. |
| **Outputs** | Content-addressed versions, dependency graph, benchmarks, provenance, and activation statistics. |
| **Method** | Immutable versions plus explicit supersession; compactness and merge/prune analysis. |
| **Failure mode** | Revoked dependencies or failed canaries disable dependants; no silent fallback to unsafe behavior. |
| **Benchmark** | Library utility, redundancy, coverage, survival, maintenance cost, model independence. |

| **Frontend Experience** | **IMPLEMENTED** |
|---|---|
| **Purpose** | Makes deterministic state feel conversational: project selection, plans, diffs, approvals, retries, diagnostics, and reload-safe progress. |
| **Inputs** | Canonical read model, streamed chat events, project bindings, action contracts. |
| **Outputs** | One coherent project card, exact action buttons, safe errors, visible retry and validation evidence. |
| **Method** | Typed client mapping with no inferred lifecycle success. |
| **Failure mode** | Missing bindings hide unsafe actions; typed backend errors are shown without exposing secrets. |
| **Benchmark** | Action visibility, reload correctness, stale-action handling, task completion time, user comprehension. |

**Engineering direction**
Progressively decompose the current frontend monolith into domain components and hooks without changing canonical lifecycle authority.

Chapter 7

# The Procedural Intelligence Compiler

The research core compiles evidence, experience, procedures, applicability, failures, and measured performance into governed procedural artifacts.

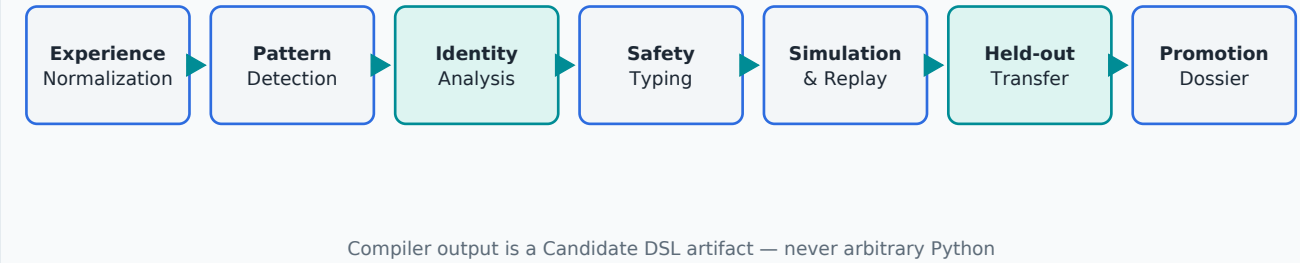



Figure 7.1 — Candidates pass through typing, identity analysis, simulation, replay, transfer, and governed promotion.

## 7.1 Scope and compiler inputs

The Procedural Intelligence Compiler is the umbrella research system. Its Capability Compiler is the subsystem that emits typed procedural IR. Source material is not raw chat: a compilation batch consists of immutable experiences linking pre-decision features, alternatives, interventions, exact artifacts, validation, approvals, failures, retries, user observations, and outcomes. Both positive and negative episodes are required. The compiler produces recommendations and candidate artifacts; it never grants production authority.

## 7.2 Formal capability object

> **Frozen decomposition**
> **C = (A, P, I, V)**, where A is applicability, P is the parameterised procedure, I is the invariant set and causal safety argument, and V is the independent verification contract.

| Element | Question answered | Versioning consequence |
|---|---|---|
| A – Applicability | Under which evidence-backed contexts may the capability activate? | Predicate changes require retroactive episode re-evaluation. |
| P – Procedure | Which typed operations and data dependencies perform the work? | A causal procedure change may require a new identity, not a minor revision. |
| I – Invariants | Why should scope, preservation, authority, and safety remain valid? | An invariant break is veto-grade negative evidence. |
| V – Verification | Which independent checks establish an acceptable result? | Verifier changes invalidate prior transfer claims until recomputed. |

The IR is a typed directed acyclic graph over trusted atomic operations. Parameters represent paths, symbols, signatures, framework objects, and bounded model slots. Evidence contracts state what must be known and fresh. Authority contracts state what remains separately approved. A model slot may generate a small fragment, but its output remains untrusted until P, I, and V are satisfied.

```
capability add_fastapi_endpoint@1
  identity:
    procedure_family: typed_route_schema_test
    invariant_family: preserve_auth_scope_and_registration
    verification_family: focused_api_contract_v2
  applicability:
    language == python
    framework == fastapi
    router_registration == present
    test_client == present
  procedure:
    router <- locate_router(route_path)
    schema <- construct_typed_schema(contract)
    endpoint <- insert_typed_endpoint(router, schema, contract)
    test <- add_focused_api_test(endpoint, schema)
  invariants:
    approved_paths_only
    preserve_authentication
    exact_symbol_and_registration
  verification:
    child_validators
    integration_names_match
    focused_test_fails_before_and_passes_after
```

## 7.3 Capability identity and evolution

One capability may span different parameterised implementations only while they share one causal procedure family, one safety argument, and one verification contract. Source similarity, task labels, trajectory embeddings, or coincidentally similar patches are insufficient. Applicability predicates may select and parameterise P. They may not encode an alternative P inside increasingly elaborate conditions.

| Decision | Use when | Required output |
|---|---|---|
| Refine applicability | P, I, and V remain one causal account but the activation region was too broad or incomplete. | New predicate version; recomputed historical membership, coverage, reliability, and transfer. |
| Split capability | A minimal witness shows a different P, I, or V is required. | Probationary child plus distinguishing witness; parent remains authoritative until evidence accumulates. |

| Decision | Use when | Required output |
|---|---|---|
| Compose capabilities | Existing capabilities form a reusable higher-order workflow. | Composite contract, integration verifier, dependency lifecycle rules, and independent benchmark. |
| Unresolved | Current vocabulary or bounded search cannot decide identity. | Preserve hypotheses and counterexamples; do not force merge or split. |

**Unification rule**
Successful unification is evidence about shared structure. Failed bounded unification is evidence about the limitations of the current compiler. It leaves identity unresolved or creates, at most, a provisional split candidate.

## 7.4 Compilation passes and epistemic outputs

**1. Experience normalization:** Preserve raw features, versions, alternatives, interventions, failures, and evidence identity.

**2. Pattern detection:** Find recurring operation and dependency structures across independent episodes.

**3. Bounded anti-unification:** Propose typed parameters without treating search failure as a split decision.

**4. Identity analysis:** Compare causal P, I, and V; emit refine, split-candidate, compose, merge-candidate, or unresolved.

**5. Applicability inference:** Identify evidence that separates valid activation, correct abstention, and false applicability.

**6. Evidence-contract inference:** Determine which observations were necessary and which vocabulary version expressed them.

**7. Safety and authority typing:** Reject non-allowlisted operations, hidden authority, unsafe paths, and unverifiable model slots.

**8. Vocabulary adequacy check:** Detect repeated ambiguity caused by an atomic operation that is too coarse.

**9. Simulation and mutation:** Exercise boundary conditions, invariant attacks, and generated counterexamples.

**10. Historical replay:** Re-run against immutable snapshots and recompute membership under current A.

**11. Held-out transfer:** Test unchanged C on uninvolved contexts with declared changed dimensions.

**12. Promotion dossier:** Freeze versions, hashes, witnesses, dependencies, metrics, thresholds, and limitations.

## 7.5 Composite capability contract

> **Emergent verification**
> **V(composite) = union of child verification obligations + V(integration).** Passing every child verifier is insufficient when names, schemas, registrations, imports, or lifecycle assumptions disagree.

| Dependency condition | Composite consequence |
|---|---|
| Any child is probationary | Composite is at most probationary. |
| Any child is degraded | Block automatic selection; allow explicit research replay only. |
| Any child is revoked | Disable the composite immediately. |
| Child version changes P, I, or V | Invalidate integration assessment until replayed and benchmarked. |

## 7.6 Lifecycle, probation, and distinguishing witnesses

| State | Meaning | Permitted use |
|---|---|---|
| Observed pattern | Repeated structure detected. | Research analysis only. |
| Candidate | Typed C=(A,P,I,V) exists. | Simulation only. |
| Probationary child | Proposed split with a minimal distinguishing witness. | Targeted replay and held-out transfer only. |
| Replay-verified | Historical snapshots and recomputed predicate membership pass. | Held-out benchmark only. |
| Experimental | Minimum transfer, precision, and safety thresholds pass. | Shadow or explicitly opted-in projects. |
| Production | Independent reuse and integration obligations pass. | Normal strategy candidate. |
| Degraded | Canary, dependency, or live reliability declined. | No automatic selection. |
| Deprecated | Superseded or no longer useful. | Replay and audit only. |
| Revoked | Unsafe, invalid, or dependency compromised. | Never executable. |

A distinguishing witness records the smallest context pair for which the parent and proposed child require different procedure, invariant, or verification behavior. A child earns independent identity only through repeated success in its region, repeated parent failure or invariant-breaking exceptions, held-out transfer, and a surviving witness. Mere clustering does not create identity.

## 7.7 Operation-vocabulary evolution

Compiler decisions depend on the granularity of trusted atomic operations. An operation such as insert_code may conceal causally different actions such as append_top_level_symbol and insert_class_member. Repeated unresolved identity, verifier branching, or invariant ambiguity inside one operation is evidence that the vocabulary needs human refinement. The compiler records a vocabulary-revision request; it cannot silently add or redefine trusted operations. Every experience records the operation-vocabulary version.

## 7.8 Why generated Python is excluded

Allowing the compiler to write arbitrary Python into Astra’s trusted kernel would make the learner an authority escalation mechanism. A restricted DSL keeps the attack surface finite and makes static analysis, replay, provenance, dependency tracking, and revocation possible. New atomic operations may be added only through reviewed engineering work and the canonical project-control path.

Chapter 8

# Experience, Memory, and Learning Algorithms

Astra learns decisions and procedures from outcomes. It does not confuse an archive of conversations with intelligence.

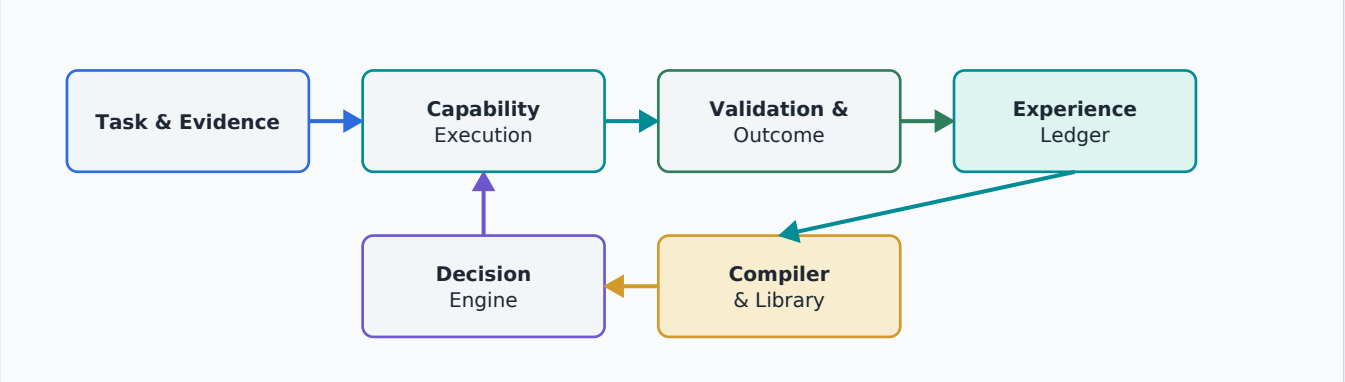



Figure 8.1 — Outcome-grounded continual improvement loop.

## 8.1 What counts as experience

An experience is not a successful edit and not a chat transcript. It is an immutable episode joining the pre-outcome repository profile, raw feature values available at decision time, evidence and operation vocabulary versions, alternatives shown, selected strategy and capability versions, intervention, exact execution and validation records, user observations, terminal outcome, and derived attribution hypotheses. Rejected plans, presentation failures, clarification conversations, failed retries, abstentions, and invalid evaluations are experiences when their provenance is complete.

> **Ownership rule**
> Outcome history belongs to immutable experiences, not permanently to the capability version that happened to process them. Capability performance is a recomputable projection.

## 8.2 Distinct stores and authority

| Store | Purpose | Mutation policy |
|---|---|---|
| Experience ledger | Canonical episodes, observations, context, interventions, and consequences. | Append-only; corrections are new records. |
| Active memory view | Task-relevant retrieval over decisions and repository conventions. | Recomputed and rankable; entries can become inactive. |
| Preference quarantine | Confirmed, scoped, testable output constraints for the local user. | Derived and deletable; excluded from capability compilation and export. |
| Capability library | Executable procedural artifacts that survived promotion. | Versioned; superseded, deprecated, or revoked explicitly. |

## 8.3 Retroactive applicability evaluation

When C@2 refines the applicability predicate of C@1, Astra must neither inherit every parent episode nor discard all history. It re-evaluates the stored pre-outcome evidence of each episode under A@2 and inherits exactly those episodes that satisfy the new predicate. This requires raw feature values, extractor identity, evidence-vocabulary version, profile version, and missingness to remain available. Coverage, reliability, transfer, promotion, and degradation statistics are then recomputed.

## 8.4 Rejection attribution and interventions

> **Attribution rule**
> **Feedback is an observation. Attribution is a hypothesis. Intervention and repeated evidence justify learning.** A selected rejection reason is not ground truth.

| Hypothesis | Possible intervention | Evidence status |
|---|---|---|
| Intent error | Re-parse or ask a targeted clarification before planning. | supported / contradicted / unresolved |
| Plan error | Re-plan with corrected scope or dependencies. | supported / contradicted / unresolved |
| Preference mismatch | Confirm a visible scoped output constraint. | supported / contradicted / unresolved |
| Context change | Refresh repository evidence; treat as a new episode if material. | supported / contradicted / unresolved |
| Interaction cost | Reduce batch size or approval burden. | supported / contradicted / unresolved |
| Presentation failure | Re-narrate the unchanged valid plan. | supported / contradicted / unresolved |

Typed rejection menus are measurement instruments and must be calibrated. They require a free-text Other option, preservation of alternatives shown, comparison with subsequent behavior, and downgrade of unreliable session-level reason selection. Numeric attribution probabilities are prohibited until a calibration dataset and reliability analysis exist.

## 8.5 Preference quarantine

| Rule | Normative consequence |
|---|---|
| Testable constraint only | Record output constraints such as patch size, narration detail, or test style; never infer psychological traits. |
| Lowest authority | Preferences cannot override evidence, repository conventions, safety, scope, approval, or verification. |
| Confirmation before influence | First proposed use is visible and requires confirmation before graduating to configuration. |
| Explicit scope | Scope is this repository, all repositories, or this task family; N=1 observation cannot establish a general trait. |
| Compilation quarantine | Preference records are excluded from capability synthesis, benchmark export, and public datasets. |
| Deletion semantics | Raw private observations remain in the append-only ledger; deleting preference means stop deriving and stop applying the view. |

## 8.6 Strategy ranker

The first learned production model should be small and transparent. Candidate features include task family, repository framework, evidence completeness, number of affected files, available semantic operations, historical strategy outcomes, model availability, and validation cost. Logistic regression or gradient-boosted trees provide strong baselines. A contextual bandit becomes useful when Astra must balance known strategies with bounded exploration, but exploration must never cross hard safety constraints.

## 8.7 Retrieval ranker

Retrieval should begin with exact paths, symbols, imports, test links, and BM25. A learned reranker may then use outcome evidence to reorder candidates. Labels should reflect whether an artifact was actually used in a successful decision, not merely whether it appeared in the prompt. The benchmark must include relevant and irrelevant near-neighbor files to measure false confidence.

## 8.8 Memory utility model

Dynamic memory is a ranking problem over immutable evidence. Features include recency, reuse count, repository similarity, task similarity, outcome quality, contradiction, and marginal contribution to a successful plan. A memory can leave the active view without being deleted. This separation permits later recomputation if the ranker was biased.

## 8.9 Failure predictor

Failure DNA should be structured into a taxonomy: wrong path, stale binding, missing import, wrong symbol, schema failure, semantic loss, test failure, insufficient evidence, provider failure, resource rejection, and user rejection. The predictor estimates likely failures before generation and selects preventive validators or alternative strategies. It remains advisory; detected real failures are determined by guards and tests.

## 8.10 Confidence without theater

A confidence percentage is meaningful only if calibrated. Astra should report hard evidence separately from predicted success. Syntax valid and tests passed are observations. A 72% probability of held-out success is a model estimate. Calibration should be measured with reliability curves, Brier score, and expected calibration error. No predicted probability converts a failing check into a pass.

## 8.11 Learning vague intent

A request such as ‘make the API architecture better’ can be handled progressively. Astra detects repository symptoms, retrieves prior decisions, proposes bounded interpretations, asks a targeted question, and records which interpretation the user selected. Over time it learns a repository-specific vocabulary and a clarification policy. It does not pretend that a small classifier has open-ended human understanding.

Chapter 9

# Repository Intelligence and Evidence

For coding systems, structure is often more valuable than more prose. Astra constructs a compact architectural map before invoking a language model.

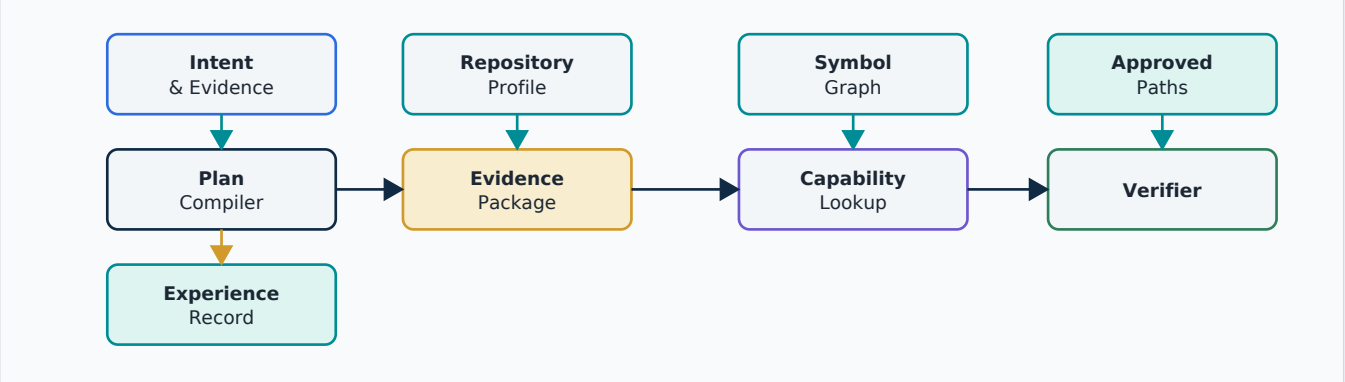



Figure 9.1 — Example task-specific repository graph and evidence package.

## 9.1 Repository profile

A repository profile is a versioned collection of measurable facts: languages, frameworks, package managers, test runners, typing mode, directory conventions, import topology, route registration, schema patterns, formatting tools, and approved validation recipes. The profile is not a personality metaphor. Every attribute must identify its extractor, evidence artifact, freshness, and confidence class.

## 9.2 Evidence package

```
EvidencePackage
  identity:
    project_run_id
    repository_root_fingerprint
    manifest_id
    content_hash

  task:
    intent_id
    requested_symbols
    approved_paths

  structure:
    target_symbols
    callers
    imports
    tests
    configuration

  source:
    bounded exact excerpts
    file_sha256
    source_spans

  limits:
    maximum_files
    maximum_characters
    exclusions
    completeness
```

## 9.3 Context compression

Compression is not an SLM summary of a hundred files. It is the deterministic reduction of a repository to the symbols, dependencies, tests, configurations, and conventions that could influence the task. Language summarization may be added after this reduction for human readability, but the structural package remains authoritative.

## 9.4 Retrieval stages

1. Exact binding: selected project, explicit path, mentioned symbol, known test.
2. Structural expansion: callers, callees, imports, registration sites, configuration.
3. Sparse ranking: lexical and BM25 relevance over bounded project artifacts.
4. Outcome reranking: previous evidence utility conditioned on task and repository profile.
5. Optional compact embedding reranking: used only when it beats the deterministic baseline.
6. Evidence validation: identities, freshness, completeness, and scope are rechecked.

## 9.5 Relevant research

RepoGraph reports that repository-level graph guidance improves multiple software-engineering methods [R3]. Repository Intelligence Graph argues for deterministic, evidence-backed architectural maps and reports accuracy and efficiency gains across coding assistants [R4]. Repository-level neural code search shows that sparse retrieval followed by neural reranking can improve file localization [R5]. Astra adopts these as design evidence, not as proof that its own implementation will achieve the same gains.

Chapter 10

# Safety, Approval, and Self-Modification

Astra may improve itself only through the same canonical project path used for any other repository, with stricter protected-module policy.

## 10.1 Authority matrix

| Component | May propose | May validate | May approve | May mutate |
|---|---|---|---|---|
| SLM | Yes, bounded | No | No | No |
| Ranker | Select allowlisted candidate | No | No | No |
| Capability compiler | Candidate DSL artifact | May request simulation, replay, and benchmark evaluation through the deterministic verifier | No | No |
| Deterministic verifier | No | Yes | No | No |
| User/authorized actor | Yes | May supply manual evidence | Yes | Through canonical command |
| Project worker | No | Executes approved validation recipes; does not determine acceptance | No | Only after exact authority |
| ProjectControlPlane | No | Accepts fresh evidence | Enforces grants | Authorizes transition |

## 10.2 Automatic focused testing

Focused testing may be automatic because Astra derives the command from a deterministic validation recipe, not from unconstrained model text. The command runs against a disposable network-disabled snapshot with no host secrets, package installation, or host fallback. CPU, memory, time, process count, and output are bounded. The exact command and result remain visible. Patch application remains a separate explicit approval.

## 10.3 Self-update policy

Astra can work on its own repository, but self-update does not mean self-authority. Small changes pass through plan approval, patch approval, isolated validation, immutable artifacts, and post-apply verification. Modules that implement approval, workspace safety, artifact integrity, isolation, or verification are protected. Changes to protected modules require stronger human review and cannot be generated by a learned capability in unattended mode.

## 10.4 Threats specific to compiled capabilities

| Threat | Required control |
|---|---|
| Overgeneralization | A procedure learned from one framework activates in a superficially similar but incompatible repository. |
| Poisoned experience | A successful outcome contains an unsafe or accidental workaround. |
| Benchmark leakage | Candidate construction uses examples that later appear in evaluation. |
| Capability explosion | Thousands of narrow procedures increase ambiguity and maintenance cost. |
| Authority smuggling | A candidate encodes a command or path outside the trusted operation vocabulary. |
| Self-confirming policy | The router repeatedly selects one strategy and never gathers evidence about alternatives. |
| Dependency drift | A trusted atomic operation changes semantics while compiled dependants remain active. |

## 10.5 Fail-closed principle

> **Non-negotiable**
> Provider unavailable, GPU busy, insufficient memory, stale evidence, invalid schema, malformed model output, unknown DSL, failed replay, out-of-scope path, missing approval, and isolation failure all remain typed non-success outcomes. Learning optimizes inside these boundaries; it does not reinterpret them.

Chapter 11

# Experimental Framework

The project becomes scientific only when baselines, chronological splits, negative outcomes, ablations, and falsification criteria are defined before results are known.

## 11.1 Baseline ladder

| System | What it contains | Purpose |
|---|---|---|
| B0 Static deterministic | Rules, AST operations, fixed capability catalog, no model. | Measures non-neural competence. |
| B1 Static + SLM | B0 plus bounded local synthesis. | Measures model contribution. |
| B2 Memory only | B1 plus retrieval of prior outcomes/workflows. | Separates recall from learning. |
| B3 Strategy ranker | B2 plus learned selection among fixed capabilities. | Measures policy learning. |
| B3.5 Prompt-guidance control | Same discovered procedures rendered as prompt guidance instead of deterministic execution. | Isolates typed activation from textual skill libraries (CODESKILL / SWE-Skills-Bench contrast). |
| B4 Capability compiler | B3 plus promoted compiled procedures. | Tests the central contribution. |
| B5 Model swap | B4 with a different or absent SLM. | Tests model independence. |

## 11.2 Task families

- Static defect detection and deterministic repair.
- Add one Python function while preserving existing symbols.
- Modify one existing function under behavioral tests.
- Add or extend focused pytest tests.
- FastAPI route, schema, service, and test additions.
- Configuration changes with typed parsers.
- Known diagnostic repair from Ruff or Pyright.
- Small multi-file refactors with explicit symbol boundaries.

- Ambiguous architectural requests requiring clarification.
- Repository-specific repeated workflows suitable for capability compilation.
- Adversarial out-of-scope, stale-binding, and malicious capability cases.

## 11.3 Chronological protocol

**1.** Freeze model, prompts, atomic capabilities, compiler version, hardware policy, and safety kernel.

**2.** Order engineering episodes chronologically by repository evolution.

**3.** Reserve future episodes as held out before synthesis, vocabulary design, verifier design, predicate refinement, and threshold selection.

**4.** Run every system variant with identical task evidence and resource limits.

**5.** Record all candidate attempts, rejections, clarifications, model calls, validation results, and user decisions.

**6.** Compile only from past episodes; never expose future outcomes to the learner.

**7.** Evaluate periodically, reporting both episode count and independent repository count: at 0, 100, 250, 500, 750, and 1,000 episodes.

**8.** Repeat with capability compilation disabled to detect whether gains come from unrelated code changes.

## 11.4 Capability-relative transfer

> **Formal definition**
> Transfer is successful reuse of an unchanged capability on a held-out episode whose capability-relevant context profile differs along declared dimensions, while procedure P, invariants I, and verification V remain valid.

**d_C(Rs,Rt) = d(pi_C(Rs), pi_C(Rt))**. Transfer distance is computed on the projection of repository features relevant to A, P, I, or V, not on an undifferentiated repository embedding. For a Python-symbol append capability, import topology, symbol location, container structure, typing mode, and test convention may matter while business domain may not.

| Stratum | Meaning | Example displacement |
|---|---|---|
| 0 – Repetition | No meaningful capability-relative displacement. | Same repository pattern and context. |
| 1 – Intra-repository | Different context within one repository. | Different package or architectural region. |
| 2 – Cross-repository ecosystem | New repository, same language/framework ecosystem. | FastAPI repository A to FastAPI repository B. |
| 3 – Cross-framework | Same language, different framework. | FastAPI route procedure to Flask-compatible abstraction. |
| 4 – Cross-language | Same procedural abstraction, different language. | Typed handler workflow across Python and TypeScript. |
| 5 – Cross-domain structural | Different domain and ecosystem with shared causal structure. | Highest and most difficult claim. |

The stratum is only a summary. Every scientific claim also records the changed-dimension signature, projection version, source and target profiles, and capability version.

> **Cross-framework qualification**
> Stratum 3 applies only when the unchanged capability is intentionally represented above framework-specific operations and its original procedure, invariants, and verification contract remain valid. Translating a FastAPI-specific procedure into a new Flask-specific procedure is adaptation or capability evolution, not transfer of the unchanged capability.

## 11.5 Held-out provenance and transfer outcomes

A target is held out only if it influenced none of capability synthesis, predicate refinement, operation vocabulary, verifier design, parameter tuning, split/merge decisions, benchmark thresholds, or promotion policy. Once used for adaptation, it cannot remain transfer evidence for the adapted version.

| Outcome | Interpretation |
|---|---|
| Transfer success | Capability activates and P, I, and V hold. |
| Correct abstention | A rejects an unsuitable context; this is successful boundary recognition. |
| False applicability | A activates where the capability should not apply. |
| Procedural failure | P fails despite valid applicability. |
| Invariant failure | I is violated; veto-grade safety or preservation evidence. |
| Verification failure | V cannot establish acceptance or exposes an integration defect. |
| Invalid evaluation | Infrastructure, oracle, leakage, or provenance prevents a capability conclusion. |

## 11.6 Promotion experiment

```
for candidate in compiler.detect_patterns(history_before_cutoff):
    ir = compiler.abstract(candidate)
    if not safety_typecheck(ir):
        reject("unsafe_ir")
    replay = evaluate(ir, historical_snapshots)
    simulation = evaluate(ir, generated_counterexamples)
    held_out = evaluate(ir, held_out_repositories)
    if promotion_policy.accepts(replay, simulation, held_out):
        publish_versioned_experimental_capability(ir)
```

## 11.7 Statistical treatment

Task success should be reported with confidence intervals and paired comparisons because each system variant attempts the same cases. McNemar-style paired analysis is suitable for binary outcomes; bootstrap intervals can summarize latency and model-call differences. Chronological transfer and forgetting require separate curves. The unit of analysis must be the independent task or repository, not every validator check inside one task.

Because multiple episodes may originate from the same repository, uncertainty estimates must account for repository-level clustering. Results should report task-level paired comparisons, repository-clustered confidence intervals, repository-disjoint transfer performance, and task-family-stratified outcomes. Episode

count must not be presented as independent sample count when episodes share a repository, template, or lineage.

## 11.8 Falsification criteria

> **Reject or revise the hypothesis if**
> compiled capabilities do not improve held-out success over memory/ranking baselines; applicability errors increase unsafe proposals; gains disappear under model swap; maintenance and regression costs exceed saved model work; or apparent gains result from benchmark leakage, task duplication, or capability-count inflation.

Chapter 12

# Metrics and Capability Growth

Capability Growth Rate is useful inventory, but intelligence must be measured by transferable marginal utility, not by accumulating procedural files.

## 12.1 Core outcome metrics

| Metric | Definition | Why it matters |
|---|---|---|
| Task success | Acceptance tests and canonical verification pass. | Primary utility. |
| Scope precision | Changed approved artifacts / all changed artifacts. | Controls unnecessary edits. |
| Model dependence | Model calls, generated characters, GPU seconds. | Tests resource efficiency. |
| Human burden | Clarifications, approvals, corrections, review time. | Measures practical usability. |
| Coverage(C) | P(A_C(e) = true) over the evaluation set. | Separates breadth from conditional success. |
| Reliability(C) | P(Success \| A_C(e) = true). | Prevents narrow predicates from looking like intelligence growth. |
| Transfer | Capability-relative reuse with stratum and changed-dimension signature (Ch 11.4). | Distinguishes learning from memorization. |
| Forgetting | Loss on previously solved held-out families. | Measures stability. |
| Safety regression | Invalid activation or blocked unsafe proposal rate. | Must not worsen. |

## 12.2 Coverage versus reliability

A narrow, highly reliable capability and a broad, moderately reliable capability make different claims. Reports must publish both **Coverage(C) = P(A_C = true)** and **Reliability(C) = P(Success | A_C = true)**. Predicate narrowing that raises reliability while silently collapsing coverage is not counted as intelligence growth.

## 12.3 Versioned transfer assessments

Transfer strata are derived views over immutable evidence packages. Every assessment records evidence-vocabulary version, capability version, profile projection, changed-dimension signature, stratum, the original assessment, the current recomputation, and `assessment_status: current | superseded`. When the evidence vocabulary changes, historical classifications are recomputed rather than silently inherited. Superseded assessments remain auditable.

## 12.4 Capability Growth Rate

**CGR = newly verified production-capability versions / completed chronological projects.** CGR reports how quickly the library changes, but it does not say whether the new capabilities are useful. A high CGR may indicate fragmentation, overfitting, or poor merge policy.

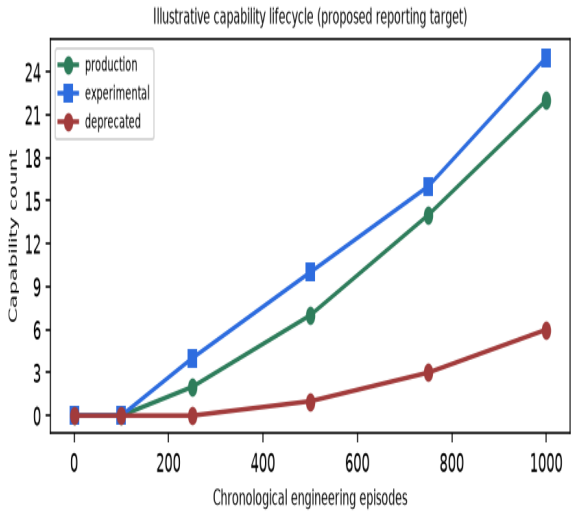



Figure 12.1 — Illustrative capability counts; the numbers are a proposed reporting example.

## 12.5 Net Verified Capability Gain

The stronger metric is Net Verified Capability Gain (NVCG). Its precise aggregation should be validated rather than chosen for convenience, but it must reward held-out coverage, applicability precision, transfer, reuse, and measured improvement while penalizing regressions, redundancy, maintenance cost, and deprecation.

```
NVCG(candidate) =
    held_out_coverage_gain
  * applicability_precision
  * transfer_success
  * marginal_success_improvement
  * independent_reuse_factor
  - regression_penalty
  - redundancy_penalty
  - maintenance_penalty
```

## 12.6 Capability dossier

| Field | Example |
|---|---|
| Identity | add_fastapi_endpoint@1 / content hash |
| Source range | Projects 120-344, temporal cutoff recorded |
| Dependencies | locate_router@2, insert_endpoint@1, add_pytest@3 |
| Applicability | FastAPI + APIRouter + Pydantic + TestClient |
| Replay | 48/50 historical snapshots |
| Held-out | 18/20 repositories; 0 scope violations |
| Marginal gain | +14 percentage points over best fixed strategy |
| Canaries | router registration, auth preservation, response schema |
| Lifecycle | experimental -> production -> degraded -> deprecated |

## 12.7 Reporting capability growth honestly

Every graph must distinguish observed results from illustrative targets. Counts should be accompanied by activation precision, reuse distribution, redundancy, deprecation, and held-out marginal gain. A library that stabilizes at 25 broadly useful procedures may be more intelligent than one that grows to 500 brittle scripts.

Chapter 13

# The Astra Research Laboratory

Production Astra must remain stable while Algorithm Lab explores ranking, compilation, simulation, and new mathematical ideas against reproducible snapshots.

## 13.1 Separation of concerns

| Production Astra | Algorithm Lab |
|---|---|
| Only promoted capabilities. | Candidate DSL and experimental compiler passes. |
| Reviewed configuration and pinned runtime. | Offline replay and generated counterexamples. |
| No training during user workflow. | Short bounded local training where appropriate. |
| Canonical safety and approval boundaries. | Cannot grant production authority. |
| Stable migrations and compatibility. | Disposable datasets, models, and metrics. |

## 13.2 Laboratory artifacts

- Immutable project snapshots and synthetic repository fixtures.
- Temporal outcome datasets with success and failure examples.
- Feature definitions and data-quality reports.
- Baseline model cards for rankers and predictors.
- Compiler versions and candidate-capability provenance.
- Replay reports, counterexample suites, and held-out benchmark results.
- Promotion recommendations that production may accept or reject.

## 13.3 Hardware-compatible experiments

The laptop is sufficient for AST and graph experiments, BM25 retrieval, SQLite analytics, logistic regression, random forests, gradient-boosted trees, small embeddings on CPU, contextual-bandit simulation, and bounded local-model inference. Long fine-tuning and architecture training remain operator-run activities and are not required for the first research phases.

## 13.4 Future mathematical research

Once the procedural baseline is stable, Astra can investigate new representations for repository state, successor-style predictions over capability graphs, uncertainty-aware routing, program anti-unification, Bayesian applicability estimation, and compact learned value functions. The correct order is empirical: first define the decision, baseline, data, and metric; then ask whether a new algorithm beats established methods on this machine.

## 13.5 Reproducibility

Every experiment should record source commit, dirty-state fingerprint, configuration, random seeds, task cutoff, model tag, provider version, hardware snapshot, container image digest, compiler version, capability library hash, and metric implementation version. An unrepeatable improvement is not a promotable result.

Chapter 14

# Implementation Roadmap

The next generation should be delivered as small, reviewable releases that first consolidate authority, then instrument outcomes, then introduce learning in shadow mode.

## 14.1 Phase plan

| Phase | Objective | Exit criterion |
|---|---|---|
| A – Definition | Freeze Astra v1 scope, canonical owners, terminology, and research contracts. | Approved architecture decision record and subsystem inventory. |
| B – Consolidation | One model boundary, workflow, retrieval path, worker, and frontend state source. | Legacy paths cannot mutate canonical state or call providers directly. |
| C – Instrumentation | Complete decision/outcome linkage and failure taxonomy. | At least 95% of benchmark decisions produce valid outcome records. |
| D – Semantic capabilities | Harden typed Python operations and focused validation. | Operation-level preservation and adversarial scope tests pass. |
| E – Rankers | Shadow retrieval, strategy, failure, and clarification models. | Calibrated held-out gains with zero authority changes. |
| F – Compiler prototype | Offline pattern -> candidate IR -> replay pipeline. | Produces candidates but no production activation. |
| G – Promotion | Held-out benchmark, canaries, lifecycle, and capability registry. | First experimental capability passes predefined thresholds. |
| H – Continual study | Chronological 100/250/500/1000-episode evaluation. | Peer-reviewable dataset, results, limitations, and ablations. |

## 14.2 Immediate engineering priorities

**1.** Create a canonical subsystem inventory and classify core, supporting, experimental, legacy, or separate product.

**2.** Route every active model invocation through LocalAIService.

**3.** Reduce main.py to application composition and split App.tsx into domain components.

**4.** Finish intent, decision, and outcome schemas with immutable artifact linkage.

**5.** Strengthen append_python_symbol to enforce exact symbol count, name, signature, and top-level-node policy.

**6.** Define automatic focused-test recipes as deterministic capability metadata.

**7.** Expand benchmark negatives, cross-repository cases, and chronological outcome fixtures.

**8.** Implement the capability DSL parser and static type checker before pattern mining.

## 14.3 What should not happen next

- Do not add another general orchestrator.
- Do not introduce more direct model adapters outside LocalAIService.
- Do not build many specialist neural models without comparative benchmarks.
- Do not enable unrestricted autonomous shell execution.
- Do not train or fine-tune as a substitute for fixing evidence and capability design.
- Do not let a candidate capability modify the safety kernel.
- Do not perform a single massive repository rewrite.

## 14.4 Product boundary recommendation

Astra v1 should be a local Python coding assistant. Assignment management, client engagement, general document automation, training pipelines, and broad specialist experiments should be separated from the active coding product path unless they directly support the research benchmark. Narrow scope makes the capability compiler measurable.

Chapter 15

# Risks, Limitations, and Ethical Research Practice

The research is valuable only if it resists inflated novelty claims, benchmark leakage, unsafe self-modification, and confidence numbers without calibration.

## 15.1 Technical limitations

- A 1.5B model remains weak on broad architecture, unfamiliar APIs, and long multi-file synthesis.
- Deterministic capabilities cover only anticipated operation families.
- Tests are incomplete specifications; passing them does not prove full correctness.
- Repository-specific learning can overfit conventions and transfer poorly.
- Capability compilation needs enough independent trajectories; early data will be sparse.
- Chronological projects are not identically distributed, complicating causal claims.
- Local Docker and Ollama availability introduce operational variability.
- The current repository has architectural debt that can confound research measurements.

## 15.2 Novelty discipline

Astra should not claim to originate workflow memory, skill extraction, self-evolving skill banks, repository graphs, test-time candidate selection, or continual-learning evaluation. Its contribution must be stated as a specific architecture and empirical result relative to these baselines. If another system already implements the same verified compiler pipeline before publication, Astra’s value may remain in its low-resource evaluation, safety integration, DSL, datasets, or negative findings.

## 15.3 User autonomy and privacy

Because Astra operates locally, raw repositories need not leave the machine. Outcome datasets should avoid secrets and unnecessary source retention. Capability provenance must identify whether procedures came from private repositories and prevent accidental publication of proprietary patterns or code. The user must be able to inspect, disable, export, or delete derived learning artifacts subject to integrity requirements.

## 15.4 Negative results are first-class

A capability that fails to transfer, a ranker that does not beat rules, or an embedding model that loses to BM25 is a useful result. The Algorithm Lab should retain failed hypotheses and publish ablations. The goal is not to prove Astra intelligent; it is to discover which forms of procedural accumulation work under real hardware and safety constraints.

Chapter 16

# Conclusion: What Astra Is Trying to Build

Astra is not a compressed frontier model. It is a software system whose verified procedural intelligence can grow while its model remains small and replaceable.

The refined Astra project asks a better question than the original attempt to make a small model behave like a large one. It asks whether software-engineering intelligence can be externalized into deterministic structure, trusted operations, repository evidence, calibrated decisions, verified procedures, and chronological experience.

The current repository already contains the hardest foundation: canonical project control, exact approval bindings, immutable artifacts, fail-closed local-AI admission, isolated execution, deterministic analysis, project retrieval, and a large regression suite. The next step is not more breadth. It is consolidation and the careful construction of a procedural-learning research boundary.

**“Astra does not trust experience. It compiles experience into candidates and trusts only what survives verification.”**

## 16.1 Success after 1,000 projects

A successful Astra after 1,000 engineering episodes is not defined by having stored 1,000 conversations. It has a compact library of versioned procedures that solve recurring task families, accurate boundaries describing when those procedures apply, evidence contracts describing what must be retrieved, failure models that prevent known mistakes, and benchmark dossiers proving marginal utility on held-out work. It makes fewer unnecessary model calls, asks better questions, touches fewer irrelevant files, and remains safe when every learned component is disabled.

## 16.2 The research promise

**Astra Next**
A deterministic software-engineering system that continually compiles verified procedural capabilities from experience, allowing useful intelligence to accumulate independently of model size while remaining local, explainable, reversible, and subordinate to explicit human authority.

A

# Appendix: Capability DSL Sketch

A restricted declarative language is the boundary between learned procedural structure and trusted implementation.

## A.1 Design requirements

- Finite, versioned instruction vocabulary.
- Typed parameters for paths, symbols, signatures, commands, and artifacts.
- No arbitrary code evaluation or shell strings.
- Explicit applicability, evidence, scope, validation, and resource contracts.
- Static dependency and authority analysis.
- Content-addressed immutable versions.
- Deterministic compilation into trusted operation handlers.
- Replayable execution trace and revocation support.

## A.2 Expanded schema

```
capability_schema: astra.capability/v1
capability_id: add_python_function_with_test
version: 3
status: experimental

provenance:
  compiler_version: capability-compiler/0.2
  source_cutoff: 2026-11-30T00:00:00Z
  source_project_count: 46
  content_hash: sha256:...

applicability:
  all:
    - language == python
    - target_file.parse_status == valid
    - requested_symbol.absent == true
    - focused_test_runner == pytest

inputs:
  target_path: SafeProjectPath
  test_path: SafeProjectPath
  symbol: PythonIdentifier
  signature: PythonFunctionSignature
  behavior_contract: TestableBehavior

evidence:
  required:
    - target_file_identity
    - neighboring_functions
    - test_conventions
  maximum_files: 6
  maximum_characters: 24000

procedure:
  - inspect_python_module(target_path)
  - synthesize_python_function(symbol, signature, behavior_contract)
  - validate_exact_top_level_symbol(symbol, signature)
  - append_python_symbol(target_path)
  - synthesize_focused_pytest(test_path, symbol, behavior_contract)
  - validate_scope()
  - run_focused_test(test_path)
  - emit_patch_artifact()

authority:
  patch_application: explicit_user_approval
  command_execution: deterministic_validation_recipe_only
  verification: canonical_verifier_only
```

B

# Appendix: Outcome and Experience Schemas

Learning quality depends on stable structured evidence, including negative outcomes and the decisions that preceded them. Schema version: astra.experience/v1 (Phase 1 charter).

**Ownership rule**
Outcome history belongs to immutable experiences, not permanently to the capability version that processed them. Capability statistics, attribution, preferences, and transfer strata are recomputable projections.

## B.1 Experience record

```
Experience  # astra.experience/v1  (immutable)
  experience_id
  occurred_at
  project_run_id?
  conversation_id?
  task_family
  intent_id?
  intent_version?
  pre_outcome_repository_profile
  raw_feature_values{}            # enough to evaluate future predicates
  evidence_vocabulary_version
  operation_vocabulary_version
  evidence_artifact_ids[]
  alternatives_shown[]
  selected_strategy
  capability_id?
  capability_version?
  predicate_version?
  model_profile_id?
  prompt_version_id?
  intervention?                   # re-plan | re-narrate | reduce_cost | confirm_preference | ...
  execution_records[]
  validation_results[]
  user_observations[]             # rejection, clarification, correction, ignore
  stated_rejection_reason?
  terminal_outcome
  failure_code?
  held_out_provenance             # none | adaptation | transfer_candidate | disqualified
  transfer_assessment_version?
  preference_influence?           # scoped; never a trait claim
  content_hash
```

## B.2 DecisionOutcome projection

DecisionOutcome remains a lean operational projection for ranking and dashboards. It does not replace the experience package. When fields conflict, the immutable experience is authoritative.

```
DecisionOutcome  # projection over Experience
  outcome_id
  occurred_at
  source
  run_id?
  task_family?
  intent?
  strategy
  model_profile?
  context_tokens?
  outcome
  failure_reason?
  duration_ms?
  evidence_artifact_id?
```

## B.3 Attribution record

```
AttributionRecord  # derived; versioned interpretation
  attribution_id
  experience_id
  evidence_vocabulary_version
  hypotheses[]:
    cause: intent_error | plan_error | preference_mismatch
         | context_change | interaction_cost | presentation_failure
    status: supported | contradicted | unresolved
    evidence_refs[]
  selected_menu_reason?           # instrument reading, not ground truth
  free_text_other?
  calibration_notes?
  assessment_status: current | superseded
```

## B.4 Preference quarantine

```
PreferenceRecord  # derived view; deletable; excluded from compilation/export
  preference_id
  scope: this_repository | all_repositories | this_task_family
  constraint                    # testable output constraint only
  status: proposed | confirmed | declined | deleted
  first_influence_confirmed_at?
  source_experience_ids[]
  # Raw observations remain in the append-only ledger.
  # Deletion means stop deriving and stop applying this view.
```

## B.5 Capability-evolution decision

```
CapabilityEvolutionDecision
  decision_id
  parent_capability_id
  parent_version
  operation: refine | split | compose | unresolved | parameterisation_gap
  witness?                      # required for split (minimal distinguishing pair)
  child_capability_id?
  child_lifecycle: probationary_child | ...
  predicate_version_after?
  operation_vocabulary_version
  evidence_vocabulary_version
  held_out_transfer_refs[]
  assessment_status: current | superseded
```

## B.6 Transfer assessment

```
TransferAssessment  # derived; recomputed when vocabularies change
  assessment_id
  experience_id
  capability_id
  capability_version
  evidence_vocabulary_version
  profile_projection_version
  changed_dimension_signature[]
  stratum: 0..5
  outcome: transfer_success | correct_abstention | false_applicability
         | procedural_failure | invariant_failure | verification_failure
         | invalid_evaluation
  original_assessment_id?       # retained when superseded
  assessment_status: current | superseded
```

## B.7 Failure taxonomy

| Class | Examples |
|---|---|
| Understanding | unknown intent, ambiguous goal, missing slot, user correction |
| Evidence | missing file, stale manifest, poor retrieval, incomplete profile |
| Strategy | wrong capability, capability gap, unnecessary model use |
| Generation | invalid JSON, wrong schema, hallucinated path, semantic mismatch |
| Scope/integrity | out-of-scope target, stale hash, altered binding, idempotency conflict |
| Validation | syntax, type, lint, test, semantic preservation, cleanup |
| Resources | gpu_busy, insufficient_vram, timeout, worker unavailable |
| Provider | provider_unavailable, model_unavailable, malformed provider response |
| Human | plan rejected, patch rejected, clarification requested, presentation failure |
| Lifecycle | cancelled, rollback, superseded, stale action |

C

# Appendix: Promotion Policy

Promotion is a scientific decision supported by a dossier, not a reward for producing a plausible-looking procedure.

## C.1 Example minimum thresholds

| Criterion | Illustrative threshold | Notes |
|---|---|---|
| Independent source projects | >= 10 | Avoid one-repository pattern extraction. |
| Historical replay success | >= 95% | Failures must be classified, not hidden. |
| Held-out repositories | >= 5 | No overlap with compilation sources. |
| Applicability precision | >= 95% | Wrong activation is costly. |
| Scope violations | 0 | Hard rejection criterion. |
| Safety regressions | 0 | Hard rejection criterion. |
| Marginal success gain | >= 5 percentage points | Versus best fixed strategy. |
| Independent production reuses | >= 3 | Required before full production status. |
| Canary pass rate | 100% | Rechecked after dependencies change. |

## C.2 Promotion decision record

Every decision records compiler version, source cutoff, benchmark version, metrics, known limitations, reviewer, activation mode, dependencies, rollback procedure, and expiry/review date. Rejection records are retained so that the compiler does not repeatedly rediscover the same invalid abstraction.

D

# Appendix: Benchmark Blueprint

The evaluation suite grows from the existing 40-case deterministic baseline into chronological, transfer, adversarial, and continual-learning studies.

## D.1 Dataset partitions

| Partition | Purpose | Leakage rule |
|---|---|---|
| Atomic fixtures | Validate trusted operations. | May be used during capability implementation. |
| Compiler discovery | Find recurring patterns. | Strictly before temporal cutoff. |
| Replay | Check historical compatibility. | Uses only snapshots already observed. |
| Development transfer | Tune thresholds. | Repositories excluded from discovery. |
| Final held-out | Primary research result. | Never used for compiler or policy tuning. |
| Adversarial | Scope, authority, poisoning, and overgeneralization. | Hidden until evaluation. |

## D.2 Required ablations

- Remove outcome failures and train from successes only.
- Replace structural repository evidence with naive text chunks.
- Use memory retrieval without compiled execution.
- Use strategy ranking without new capabilities.
- Disable applicability learning and use broad task labels.
- Allow capability count to grow without merge/prune.
- Swap or remove the SLM after compilation.
- Disable compiled capabilities while preserving all other later code.

E

# Appendix: Research Decision Taxonomy

A stable vocabulary prevents architecture discussions from collapsing different kinds of learning into the same word.

## E.1 System components

| Term | Role |
|---|---|
| Deterministic plan compiler | Composes already-available capabilities into a bounded task plan; does not invent new procedures. |
| Capability compiler | Creates candidate procedural artifacts from verified experience under a restricted DSL. |
| Semantic edit engine | Implements trusted atomic operations (locate, insert, rewrite, validate) used by capabilities. |
| Capability registry | Resolves immutable, content-addressed capability versions and dependencies. |
| Capability library | Governed collection of promoted (and lifecycle-managed) capabilities. |
| Decision engine | Selects among allowlisted strategies using rules, memory, and optional shadow/promoted rankers. |

## E.2 Research objects

| Term | Definition |
|---|---|
| Atomic capability | Human-reviewed trusted operation implemented in normal code. |
| Composite capability | Declarative composition of atomic capabilities with an integration verifier. |
| Candidate capability | Unpromoted artifact produced by the compiler. |
| Applicability model | Predicts whether a capability is appropriate for a context. |
| Evidence contract | Facts that must be present and fresh before activation. |
| Strategy | A selectable approach to a task; may invoke one or more capabilities. |
| Trajectory | Ordered decisions, operations, observations, failures, and outcomes. |
| Experience | Immutable episode joining context, alternatives, intervention, outcome, and attribution hypotheses. |
| Memory | Derived retrieval view over immutable experience. |
| Procedural intelligence | Executable, tested knowledge of how to perform a task. |
| Capability compilation | Transformation of verified experience into candidate procedural IR and promoted artifacts. |
| Safety kernel | Fixed authority, scope, integrity, approval, isolation, and verification mechanisms. |

F

# Appendix: Related Work and Positioning

Astra should be compared against deterministic repair, repository graphs, workflow memory, experience banks, skill libraries, test-time scaling, and continual-learning benchmarks.

| Work | Relevant idea | Astra positioning |
|---|---|---|
| Agentless | Simple localization, repair, and validation without an autonomous tool-planning agent. | Supports simple structured baselines; Astra adds chronological procedural compilation. |
| SWE-agent | Agent-computer interface designed for repository navigation and editing. | Supports purpose-built tool interfaces; Astra restricts authority and compiles procedures. |
| RepoGraph / RIG | Repository-level graphs and deterministic architecture maps. | Direct foundation for repository intelligence and evidence construction. |
| S* | Sequential and parallel test-time scaling with execution-grounded selection. | Supports bounded candidate retry and verifier-guided selection. |
| Agent Workflow Memory | Induces reusable workflows and retrieves them for future tasks. | Astra must distinguish executable promoted procedures from remembered workflow text. |
| SWE-Exp | Distills successful and failed repair experiences. | Supports outcome-grounded experience; Astra adds compiler IR and fixed safety integration. |
| SWE-Bench-CL | Chronological continual-learning evaluation and forgetting metrics. | Strong template for temporal splits and transfer measurement. |
| SkillFoundry | Mines validated executable skills from heterogeneous scientific resources. | Closest conceptual neighbor; Astra focuses on real coding outcomes and local constrained execution. |
| CODESKILL | Learns skill extraction and maintenance policy with reinforcement learning. | Shows skill-bank learning is established; Astra must demonstrate distinct verified compilation value. |
| SWE-Skills-Bench | Paired execution-based evaluation with and without skills. | Supports marginal-utility testing rather than capability counting. |

G

# Appendix: References

Primary papers and repository evidence used to position the proposal. URLs were verified during manuscript preparation.

**[R1]** Xia, C. S. et al. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489

**[R2]** Yang, J. et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024. https://arxiv.org/abs/2405.15793

**[R3]** Ouyang, S. et al. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. arXiv:2410.14684, 2024. https://arxiv.org/abs/2410.14684

**[R4]** Cherny-Shahar, T. and Yehudai, A. Repository Intelligence Graph: Deterministic Architectural Map for LLM Code Assistants. arXiv:2601.10112, 2026. https://arxiv.org/abs/2601.10112

**[R5]** Gandhi, S., Gao, L., and Callan, J. Repository-level Code Search with Neural Retrieval Methods. arXiv:2502.07067, 2025. https://arxiv.org/abs/2502.07067

**[R6]** Li, D. et al. S*: Test Time Scaling for Code Generation. arXiv:2502.14382, 2025. https://arxiv.org/abs/2502.14382

**[R7]** Wang, Z. Z. et al. Agent Workflow Memory. arXiv:2409.07429, 2024. https://arxiv.org/abs/2409.07429

**[R8]** Chen, S. et al. SWE-Exp: Experience-Driven Software Issue Resolution. arXiv:2507.23361, 2025. https://arxiv.org/abs/2507.23361

**[R9]** Joshi, T., Chowdhury, S., and Uysal, F. SWE-Bench-CL: Continual Learning for Coding Agents. arXiv:2507.00014, 2025. https://arxiv.org/abs/2507.00014

**[R10]** Shen, S. et al. SkillFoundry: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. arXiv:2604.03964, 2026. https://arxiv.org/abs/2604.03964

**[R11]** Li, Y. et al. CODESKILL: Learning Self-Evolving Skills for Coding Agents. arXiv:2605.25430, 2026. https://arxiv.org/abs/2605.25430

**[R12]** SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401, 2026. https://arxiv.org/abs/2603.15401

**[R13]** SkillX: Automatically Constructing Skill Knowledge Bases for Agents. arXiv:2604.04804, 2026. https://arxiv.org/abs/2604.04804

**[R14]** SkCC: Portable and Secure Skill Compilation for Cross-Framework LLM Agents. arXiv:2605.03353, 2026. https://arxiv.org/abs/2605.03353

**[R15]** Ma, Z. et al. Rethinking Verification for LLM Code Generation: From Generation to Testing. arXiv:2507.06920, 2025. https://arxiv.org/abs/2507.06920

**[R16]** Astra repository README.md, stage0 trust-integrity, stage1 control-plane, stage2 isolation, phase5-9 integration, stabilization checkpoint, and phase0 benchmark artifacts. Inspected 29 July 2026 on commit 9d7b63a41cf4.

**[R17]** Astra deterministic-core benchmark phase0.v1. Latest inspected artifact: benchmarks/results/baseline_2026-07-28.json, 40 cases, 40 passed.

# End of research blueprint

The next step is to convert this blueprint into an architecture decision record, a canonical subsystem inventory, and a controlled Phase A implementation plan.

**Final definition**
**Astra Next Research Blueprint v1.1 — Phase 1 Normative Revision**
**Astra compiles verified procedural capabilities from experience.**
Its intelligence accumulates independently of whichever language model is currently attached.